

DIA 1

O que é um agente, de verdade

Agent harness, as cinco camadas e o loop rodando na sua própria máquina

1

Por que um LLM sozinho não age

2

As cinco camadas de um harness

3

Modelos locais por quatro caminhos

4

Escrever o loop em Python puro

As oito horas de hoje

1

09:00 – 10:30 • O problema e o vocabulário

Por que o modelo inventa, o que é um harness, as cinco camadas e o loop por dentro. Termina com a turma sabendo nomear cada peça.

2

10:45 – 12:15 • Laboratório: o modelo na sua máquina

Subir Ollama, llama.cpp e vLLM. Provar que o mesmo cliente fala com os três. Medir a diferença.

3

13:15 – 14:45 • Como os outros fazem

ReAct e variantes, harnesses prontos, e quatro equipes reais que resolveram isso em produção.

4

15:00 – 16:30 • Laboratório: escrever o harness à mão

Ferramentas, schemas, o loop, a parada. Sessenta linhas que funcionam.

5

16:30 – 17:00 • Fechamento

Demos rápidas, erros clássicos, tarefa de casa.

01

0 problema

Começamos por uma resposta inventada.

Rode isto agora, na pasta de um projeto seu

Uma pergunta sobre o SEU mundo. O modelo responde com um número exato e um tom seguro.

E erra.

Não é um modelo ruim. Qualquer modelo faz isso, do menor local ao maior de fronteira.

```
$ ollama run qwen3:4b \  
  "Quantas linhas tem o README.md  
  deste projeto?"  
  
> "O README.md tem 128 linhas."  
  
$ wc -l README.md  
  
> 47 README.md
```

E agora sem a menor chance de sorte

Se a primeira ainda deixa dúvida — 'talvez ele tenha chutado bem' — a segunda não deixa nenhuma.

Hash plausível. Mensagem plausível. Autor plausível. Tudo inventado.

Repare no que ele NÃO fez: dizer que não tem como saber.

```
$ ollama run qwen3:4b \  
  "Qual foi o ultimo commit  
  deste repositorio, e quem fez?"  
  
> "a3f9c21 - fix: corrige validacao  
> de token (Maria Silva)"  
  
$ git log -1 --oneline  
  
> 8e1b402 docs: atualiza README
```

O PONTO DE PARTIDA

O modelo não tem acesso ao seu mundo. E não avisa que não tem.

Um LLM só enxerga o texto que está no contexto. Ele não abre arquivos, não lê o disco, não consulta a rede, não olha o relógio. Quando você pergunta sobre o mundo, ele não consulta o mundo: completa a frase com o que é estatisticamente plausível.

A SEGUNDA METADE DO PROBLEMA

Mesmo quando acerta, ninguém consegue provar que acertou.

A LangChain mediu isso no Terminal Bench 2.0: o modo de falha dominante é o agente terminar de escrever, reler o próprio código, achar bom e parar. Sem rodar teste, sem verificar caso de borda. Não é incapacidade — é satisfação fácil demais.

Quatro faltas, e nenhuma se resolve com prompt

O modelo sozinho não consegue...	O que falta	Onde resolvemos
OLHAR o seu mundo	Ler arquivos, bancos, APIs, repositório	Dia 1 e 2
AGIR no seu mundo	Escrever, executar, publicar	Dia 1 e 3
PROVAR que acertou	Rodar teste, comparar, verificar	Dia 4 e 5
LEMBRAR do que descobriu	Persistir entre execuções	Dia 3 e 4

Agent = Model + Harness

O harness é tudo que não é o modelo: instruções, limites, ferramentas, verificação, memória, orquestração. É o que você escreve, versiona e depura. E ele não deixa o modelo mais inteligente — deixa a inteligência do modelo útil e verificável.

A DEFINIÇÃO DA OPENAI, FEVEREIRO DE 2026

**Se o modelo de raciocínio é o cérebro,
o harness são as mãos e os pés.**

Um harness é o tool shell que permite a um agente afetar o mundo real. Ler arquivos, corrigir código, rodar testes, publicar em produção: tudo acontece dentro do harness. Repare que a definição não diz 'prompt' — diz shell, um ambiente inteiro.

Harness tem uso técnico em cinco áreas, e sempre significa o mesmo

1 Arreio

As rédeas não deixam o cavalo mais forte. Tornam a força dele aproveitável. É a metáfora que deu nome à coisa.

2 Chicote elétrico

Norma NASA-STD-8739 para naves. Um fio solto e a missão acaba. Transmissão precisa de sinal num ambiente caótico.

3 Test harness

Conceito antigo de engenharia de software: stubs e drivers que criam um ambiente controlado onde o sujeito do teste se comporta previsivelmente.

4 Cinto de segurança

Não restringe seu movimento. Impede que você caia quando escorregar. É o guardrail do agente.

5 Chicote automotivo

Não gera energia nem calcula nada. Sem ele, as peças do carro são ilhas isoladas que não se conhecem.

= O que há em comum

Nenhum substitui a força central. Todos tornam essa força controlável, confiável e útil. É exatamente o que fazemos com o modelo.

Mesmo modelo, mesma tarefa, dois harnesses

Configuração	Custo	Tempo	Resultado
Agente sozinho, sem harness	US\$ 9	20 min	Software quebrado
Harness completo	US\$ 200	6 h	Software funcionando

02

As cinco camadas

O vocabulário que o campo inteiro adotou em 2026.

As cinco camadas de um harness

Camada	Pergunta que ela responde	O que acontece se faltar
Instructions	O que a IA deve saber e seguir?	Não sabe quem é nem quais são as regras
Constraints	O que ela está proibida de fazer?	Regra vira sugestão, e sugestão é ignorada
Feedback	Como sabemos que deu certo?	O agente acha que acertou e para
Memory	O que sobrevive entre execuções?	O mesmo erro pela terceira vez
Orchestration	Como vários agentes se coordenam?	Um contexto só, e ele estoura

Instructions: o que o agente sabe antes de começar

1

Identidade e escopo

Quem é o agente, o que está no escopo dele e o que explicitamente não está. Duas ou três frases, não uma página.

2

Mapa, não manual

Estrutura do projeto e ponteiros para onde está a documentação profunda. O agente sabe navegar; ele só não sabe onde as coisas estão.

3

Formato de saída

Se você precisa de JSON, diga o schema. Se precisa de citação de fonte, diga o formato exato da citação.

4

Onde isso mora

AGENTS.md, CLAUDE.md, system prompt, arquivos em prompts/. O nome muda; a função é a mesma.

A mesma regra, escrita de dois jeitos

X Não faz nada

- Escreva código limpo e bem estruturado
- Sempre siga as melhores práticas
- Seja cuidadoso e não cometa erros
- Use boas convenções de nomenclatura
- NÃO ALUCINE

OK Muda o comportamento

- Migrations ficam em db/migrations/, nunca em app/
- Toda função pública precisa de type hints
- Datas viajam em ISO-8601 UTC entre serviços
- Nunca importe de core/ dentro de adapters/
- Se um teste falhar, pare e reporte; não desative o teste

Constraints: o que ele não consegue fazer, nem querendo

1

Sugestão x restrição

Instruction pede. Constraint impede. A primeira depende da boa vontade do modelo; a segunda roda no seu código, antes da ferramenta executar.

2

Hooks

Código seu que intercepta o ciclo. O PreToolUse é o único que pode NEGAR uma chamada de ferramenta. Formalizamos isso no dia 3.

3

Sandbox e permissões

Container, usuário sem privilégio, sistema de arquivos limitado, rede fechada. Um agente com shell é um agente que pode apagar seu disco.

4

Portões automáticos

Linter, type checker, teste no CI. O agente não passa; não é questão de ele concordar.

Feedback: como o agente descobre que errou

1

O sinal precisa ser automático

Teste que roda, linter que reclama, requisição HTTP que devolve 500. Nada que dependa de um humano ler.

2

E precisa voltar ao contexto

Um teste que falha e ninguém injeta na conversa não é feedback: é log. O agente só reage ao que ele lê.

3

Verificação independente

Quem escreveu não é quem avalia. A Anthropic separa Generator e Evaluator justamente porque o autor é mau juiz do próprio trabalho.

4

Erro útil, não erro cru

'FileNotFoundException' ensina pouco. 'arquivo não encontrado; disponíveis em docs/: a.md, b.md' faz o agente acertar na volta seguinte.

Memory: o que atravessa o tempo

1

Curto prazo é a conversa

O histórico de mensagens da execução atual. Morre quando o processo morre, e estoura quando fica grande demais.

2

Longo prazo é decisão de projeto

O que merece sobreviver? Preferência do usuário, correção que você já fez duas vezes, fato descoberto que custou caro descobrir.

3

Notas fora da janela

Arquivo em disco, banco, checkpointer. O agente escreve e relê. É como um humano usa caderno: não para lembrar, para não precisar lembrar.

4

Quem é o dono da memória

Toda chave de sessão precisa carregar o usuário. Misturar memórias de dois usuários é vazamento de dados, não bug de UX.

Orchestration: o que atravessa o espaço

1

Um contexto por especialidade

A razão principal de dividir não é paralelismo, é isolamento: cada agente carrega só o que a tarefa dele exige.

2

Papéis separados

Planejar, executar e avaliar em agentes diferentes. Quem escreveu o código é o pior juiz do código.

3

Contrato entre agentes

O que entra, o que sai, em que formato. Sem contrato explícito, multiagente vira telefone sem fio caro.

4

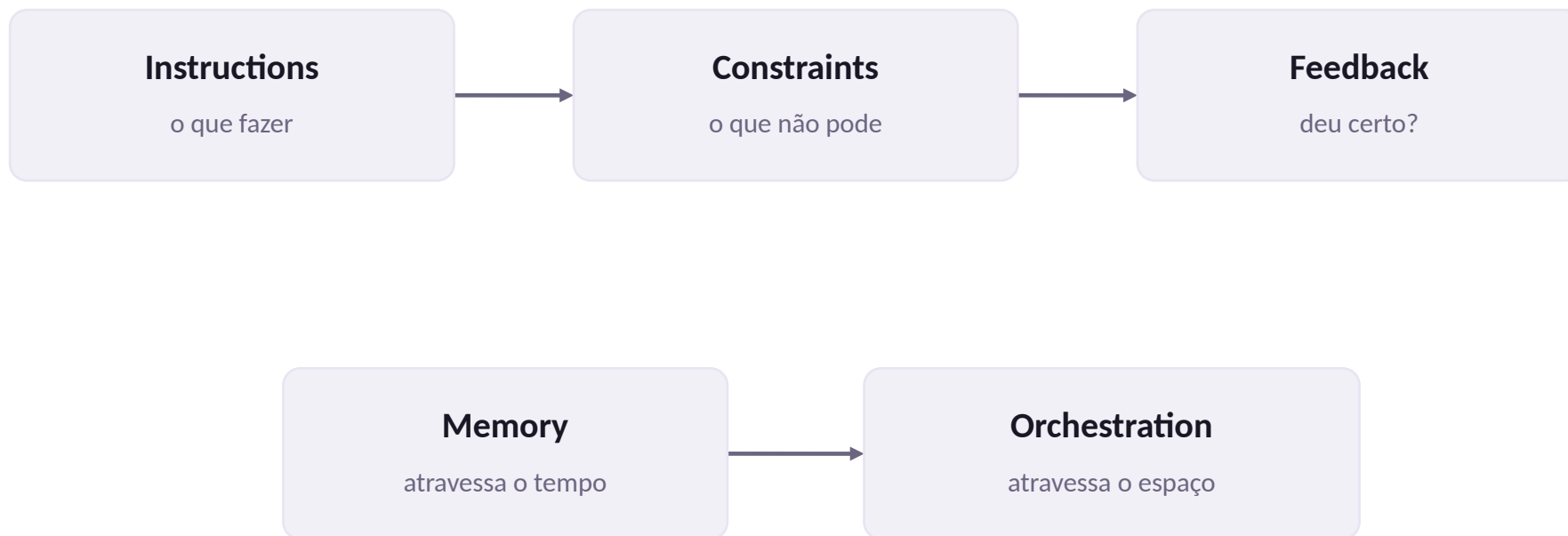
A camada mais imatura

Em 2026 esta é a parte menos assentada do campo. Comece por um agente e só divida quando doer.

Para não se perder lendo documentação de fornecedor

Cinco camadas	Quatro verbos (OpenAI)	Três blocos (Fowler)	Implementação típica
Instructions	inform	Context engineering	AGENTS.md, CLAUDE.md, system prompt
Constraints	constrain	Restrições arquiteturais	Hooks, linter, sandbox, CI
Feedback	verify	Garbage collection	Agente avaliador, testes, schema
Memory	inform	Context engineering	Checkpointing, base de conhecimento
Orchestration	correct	(não coberto)	Multiagente, pipeline, subgrafos

Antes, durante, através do tempo e do espaço



Instructions e constraints agem ANTES do agente agir. Feedback age DURANTE. Memory faz a experiência sobreviver entre sessões. Orchestration faz vários agentes trabalharem ao mesmo tempo.

Três perguntas antes do intervalo

1

Um agente repete a mesma busca seis vezes seguidas

Qual camada está faltando?

2

Um agente entregou código que não compila e disse que estava pronto

Qual camada está faltando?

3

Você corrigiu a mesma coisa em três sessões diferentes

Qual camada está faltando?

03

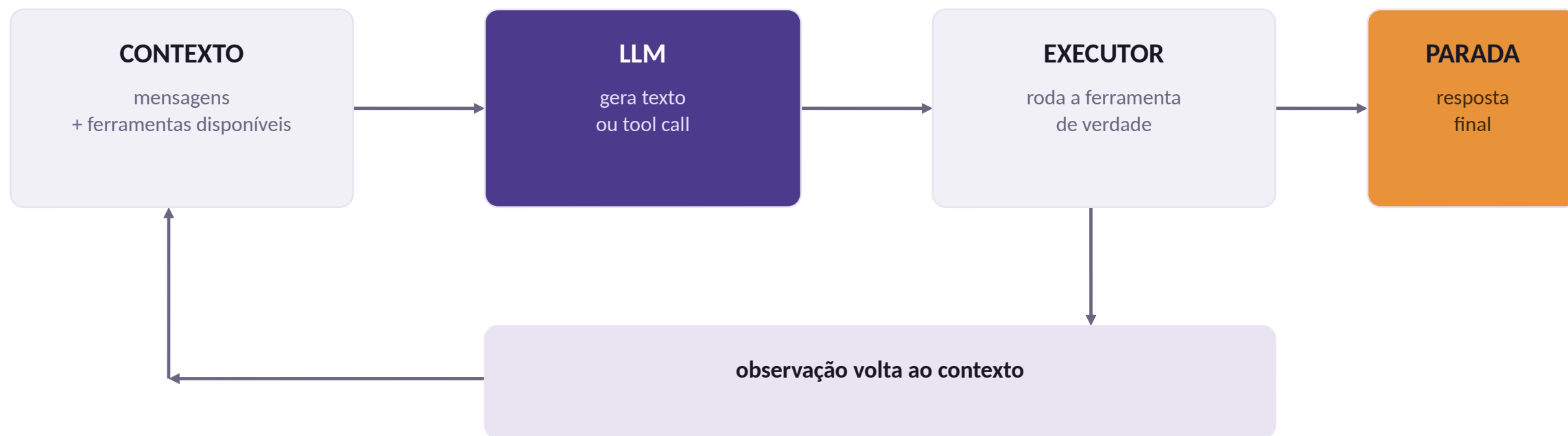
Anatomia do runtime

As camadas dizem o quê. O loop diz o como.

Os cinco passos do loop, por dentro

Pergunta	Passo do runtime	Sem isso...
O que o modelo sabe agora?	Montagem de contexto	Não sabe quem é nem o que já fez
O que o modelo pode fazer?	Registro de ferramentas	Só fala, não age
Como executo o que ele pediu?	Despachante e sandbox	Nada acontece de fato
Quando eu paro?	Condições de parada	Loop infinito ou parada precoce
O que sobra entre execuções?	Memória e estado	Cada execução começa do zero

O loop



Cada volta é um passo. Um agente que resolve a tarefa com 6 tool calls deu 7 voltas.

O que realmente trafega numa execução de dois passos

```
# volta 1 - o que sai daqui para o modelo
[ {"role": "system", "content": "Voce e um assistente. Ferramentas: ler_arquivo, calcular."},
  {"role": "user", "content": "Qual fornecedor sai mais barato em 24 meses?"} ]

# o modelo devolve: nao e texto, e uma tool call
{"tool_calls": [{"id": "c1", "name": "ler_arquivo", "args": {"caminho": "docs/fornecedores.md"}}]}

# o executor roda de verdade e o resultado volta como MENSAGEM
[ ...as duas de cima...,
  {"role": "assistant", "tool_calls": [ {"id": "c1", ...} ]},
  {"role": "tool", "tool_call_id": "c1", "content": "Fornecedor A - 1200/mes; Fornecedor B - 950/mes"} ]

# volta 2 - agora o modelo tem o dado e pede a conta
{"tool_calls": [{"id": "c2", "name": "calcular", "args": {"expressao": "950 * 24"}}]}

# volta 3 - com as duas observacoes no contexto, ele responde
"O Fornecedor B custa 22800 em 24 meses. [fonte: docs/fornecedores.md]"
```

Repare: o histórico só cresce. Cada observação vira uma mensagem de papel tool, amarrada pelo tool_call_id.

O que o harness NÃO é

1 Não é o modelo

Trocar qwen3:4b por llama3.1:8b não muda uma linha do harness. São camadas independentes, e é por isso que a equação é uma soma.

2 Não é o prompt

O prompt é dado de entrada do harness.
Reescrever o prompt não conserta um loop sem condição de parada, e nunca vai consertar.

3 Não é o framework

LangChain e LangGraph implementam partes do harness. Você pode ter harness sem framework nenhum, e é o que faremos hoje à tarde.

Quase toda falha de agente é falha de harness

Sintoma	Componente culpado	Onde mexer
Repete a mesma ferramenta 8 vezes	Condição de parada	Contador de passos e detecção de repetição
Esquece o que descobriu no passo 2	Montagem de contexto	Verificar se a observação foi anexada
Chama ferramenta que não existe	Registro de ferramentas	Nomes distintos e erro que lista as válidas
Devolve JSON quebrado	Validação	Schema mais retry com o erro reinjetado
Gasta 40k tokens numa pergunta simples	Política de contexto	Poda, resumo, ferramentas mais específicas
Para antes de terminar a tarefa	Feedback	Critério de pronto explícito e verificável

Condições de parada: quatro, não uma

A pergunta 'quando eu paro' parece trivial e é a que mais derruba agente em produção.

Parada demais entrega trabalho pela metade. Parada de menos queima orçamento em silêncio.

As quatro convivem. Nenhuma sozinha é suficiente, e a ordem em que você testa importa: limite duro primeiro, resposta final por último.

```
# 1. resposta final: o modelo nao
#   pediu ferramenta nenhuma
if not resp.tool_calls:
    return resp.content

# 2. limite duro de passos
if passo >= MAX_PASSOS:
    return "limite de passos atingido"

# 3. repeticao: mesma ferramenta,
#   mesmos argumentos, de novo
chave = (nome, json.dumps(args))
if chave in vistos:
    return "repeticao detectada"
vistos.add(chave)

# 4. orcamento de tokens
if tokens_gastos > TETO:
    return "orcamento estourado"
```

RESUMO DO BLOCO

O loop tem cinco passos, e cada bug de agente nasce em um deles.

Montagem de contexto, registro de ferramentas, despachante, parada, memória. Quando algo der errado nas próximas cinco semanas, a primeira pergunta não é qual prompt reescrever: é em qual desses cinco passos o defeito nasceu.

04

Modelos locais

Quatro caminhos para ter um modelo na sua própria máquina.

Três razões que não são ideologia

1 O dado não sai

Contrato, prontuário, código proprietário. Em vários setores essa é a única razão que importa, e ela sozinha decide a arquitetura.

2 O custo por token é zero

Um agente em loop faz dezenas de chamadas por tarefa. Em experimentação, é a diferença entre iterar à vontade e olhar o painel de custo a cada teste.

3 Você controla a versão

O modelo não muda debaixo de você numa terça-feira. Para avaliação e reprodutibilidade, isso vale mais do que alguns pontos de benchmark.

Quatro formas de servir um modelo local

Caminho	Para que serve	Ponto forte	Ponto fraco
Ollama	Dia a dia, prototipagem	Um comando e funciona; API compatível com OpenAI	Menos controle fino de parâmetros
llama.cpp	Controle fino, hardware modesto	Roda quantizado em CPU; flags para tudo	Você monta o servidor na mão
vLLM	Servir a sala inteira	Throughput alto, batching contínuo	Quer GPU e memória de verdade
Transformers	Entender o que acontece	Você vê tokenizer, logits, geração	Lento e sem servidor pronto

Um cliente, três backends

Os três servidores expõem a mesma API. Isso significa que trocar de backend é trocar duas linhas — não reescrever o agente.

Prove isso na sua máquina antes do almoço.

Critério de pronto: a mesma pergunta respondida por dois backends diferentes, com o mesmo código, e o tempo de cada um medido.

```
from openai import OpenAI

# Ollama
cli = OpenAI(base_url="http://localhost:11434/v1",
             api_key="nao-usado")
MODELO = "qwen3:4b"

# llama.cpp
cli = OpenAI(base_url="http://localhost:8080/v1",
             api_key="nao-usado")
MODELO = "local"

# vLLM
cli = OpenAI(base_url="http://localhost:8000/v1",
             api_key="nao-usado")
MODELO = "Qwen/Qwen3-8B"

r = cli.chat.completions.create(
    model=MODELO,
    messages=[{"role": "user",
               "content": "Responda em uma frase: "
                           "o que é um agent harness?"}])
print(r.choices[0].message.content)
```

Tool calling em modelo local

1 llama.cpp

```
llama-server -m modelo.gguf --jinja
```

Sem --jinja o servidor ignora o chat template embutido no GGUF, e o tool calling simplesmente não funciona.

2 vLLM

```
vllm serve Qwen/Qwen3-8B --enable-auto-tool-choice --tool-call-parser hermes
```

Sem as duas flags, a tool call volta como TEXTO dentro do conteúdo.

3 Sintoma típico

resp.tool_calls vem vazio e a resposta traz algo como <tool_call>{...}</tool_call> em texto puro. É servidor mal configurado, não modelo ruim.

Escolhendo um modelo para agente

1

Tool calling nativo e confiável

Mais importante que pontuação geral. Um modelo brilhante que emite JSON quebrado é inútil dentro de um loop.

2

Janela de contexto real

O número anunciado e o número em que ele ainda recupera bem são diferentes. Teste com o seu histórico típico, não com o do fabricante.

3

Tamanho que cabe na sua máquina

Um 8B quantizado que responde em 4 segundos vence um 32B que responde em 90. No loop, latência multiplica por número de passos.

4

Licença compatível com o seu uso

Open weights não é sinônimo de uso comercial livre. Leia antes de colocar em produto.

Laboratório da manhã: o que entregar

1

Um modelo respondendo localmente

Ollama instalado, modelo baixado, uma pergunta respondida no terminal.

2

O mesmo script falando com dois backends

Trocar `base_url` e `model`, e obter resposta nos dois. Se você só tem um backend, use dois modelos diferentes no mesmo servidor.

3

Tempo medido

Cronometre a mesma pergunta nos dois. Anote. Vamos usar esse número à tarde, quando o loop multiplicar a latência.

4

Extensão para quem terminar antes

Faça a primeira chamada com `tools=[...]` e observe o campo `tool_calls` da resposta. É o que a tarde inteira vai explorar.

05

ReAct e os padrões de loop

O artigo de 2022 que todo framework copiou.

YAO ET AL., 2022

Em vez de pedir a resposta, peça o próximo passo.

A ideia inteira do ReAct cabe nessa frase. O modelo alterna entre pensar e agir, e o harness devolve a ele o resultado real de cada ação. O ganho não vem de o modelo ficar mais inteligente: vem de ele decidir com informação de verdade em vez de decidir com suposição.

ReAct: pensar, agir, observar

O traço abaixo é o que trafegava no formato original: texto puro, com o harness fazendo parsing das linhas Action e Action Input.

Hoje os modelos emitem tool calls em campo estruturado, e o parsing sumiu. Mas o padrão lógico é idêntico.

Conhecer a versão em texto é o que salva quando o modelo local não tem tool calling nativo.

```
Thought: nao sei o conteudo do arquivo,  
        preciso ler  
Action: ler_arquivo  
Action Input: docs/fornecedores.md  
Observation: Fornecedor A - R$ 1.200/mes;  
              Fornecedor B - R$ 950/mes
```

```
Thought: agora tenho os valores,  
        preciso do total em 24 meses  
Action: calcular  
Action Input: 950 * 24  
Observation: 22800
```

```
Final Answer: O Fornecedor B custa  
              R$ 22.800 em 24 meses  
              [fonte: docs/fornecedores.md]
```

```
# o harness le Action e Action Input,  
# executa, e injeta Observation  
# de volta no contexto.
```

Texto parseado x campo estruturado

22 ReAct em texto puro

- O modelo escreve Action: e Action Input:
- O harness usa regex para extrair
- Funciona com QUALQUER modelo, até sem tool calling
- Quebra quando o modelo muda o formato
- É o plano B da tarde de hoje

26 Tool calling estruturado

- O modelo devolve resp.tool_calls, campo da API
- Os argumentos já vêm como objeto
- Schema validado antes de executar
- Exige modelo e servidor configurados para isso
- É o caminho principal do curso

Padrões de loop que vale conhecer

R ReAct

Pensar, agir, observar, repetir. Padrão de referência. Funciona quase sempre. É o que vamos escrever hoje.

P Plan-and-Execute

Planeja tudo antes, depois executa. Economiza tokens de raciocínio em tarefas longas e dá um plano legível para um humano aprovar.

F Reflexion

Depois de falhar, o agente critica a si mesmo e tenta de novo. Precisa de sinal claro de sucesso. Usaremos no dia 4.

C CodeAct

O agente escreve código Python que chama as ferramentas, em vez de JSON. Base do smolagents. Dia 5.

T Tree of Thoughts

Ramifica várias linhas de raciocínio e poda as ruins. Caro. Raramente compensa em produção.

= O que todos têm em comum

Montar contexto, decidir, agir, observar, parar. Muda a política de decisão; não muda o esqueleto.

Quando trocar de padrão vale a pena

Situação	Padrão indicado	Por quê
Tarefa exploratória, caminho desconhecido	ReAct	Decide o próximo passo com o que acabou de descobrir
Etapas previsíveis e um humano aprova antes	Plan-and-Execute	O plano é um artefato legível e aprovável
Existe teste automático de sucesso	Reflexion	O agente usa o resultado do teste para se corrigir
Ferramentas que se compõem muito entre si	CodeAct	Uma linha de Python encadeia o que seriam 5 tool calls
Não tem sinal de sucesso nenhum	ReAct com humano no meio	Sem sinal automático, o revisor é o feedback

CHECKPOINT DO BLOCO

**Se você tirar o Observation do traço,
o que sobra é o slide de abertura do dia.**

Um agente sem observação real é um modelo escrevendo uma história plausível sobre o que teria acontecido. Todo o resto do curso é sobre garantir que a observação seja verdadeira, que ela volte ao contexto, e que alguém verifique.

06

Panorama de harnesses

Agent Zero, Hermes e a sopa de letrinhas.

Agent Zero

1

O computador é a ferramenta

Em vez de quarenta ferramentas específicas, entrega ao agente um Linux dockerizado com terminal, navegador e sistema de arquivos, e deixa que ele se vire.

2

Hierarquia superior e subordinado

Um agente cria subagentes para subtarefas. O subagente tem contexto isolado e devolve um relatório. É a topologia do nosso dia 4.

3

Prompts como comportamento

Todo o comportamento vive na pasta prompts/, em arquivos de texto editáveis. Não há comportamento escondido em código.

4

Roda em Docker, sempre

Um agente com shell é um agente que pode apagar seu disco. Isolamento é requisito, não enfeite.

A LIÇÃO QUE INTERESSA

Se você não consegue ler o prompt exato que foi enviado, você não consegue depurar.

Essa é a razão de o Agent Zero manter todo o comportamento em arquivos de texto. Framework que esconde a montagem do contexto transforma depuração em adivinhação, e depurar por adivinhação é a diferença entre resolver em vinte minutos e resolver em dois dias.

Hermes: duas coisas diferentes

M Hermes, a família de modelos

- Modelos open-weights ajustados pela Nous Research
- Historicamente muito bons em function calling
- O formato de tool call virou padrão de fato
- O vLLM tem um parser chamado literalmente hermes
- É o que você escolhe no --tool-call-parser

H Hermes Agent, o harness

- Agente autônomo que roda continuamente
- Memória persistente com busca e sumarização
- Cria e melhora as próprias skills, e as reutiliza
- Vários backends: local, Docker, SSH, serverless
- Bot Mode: times de especialistas conversando

Hermes Agent também fala e escuta

TTS Texto vira voz

Onze provedores possíveis, locais e gratuitos (Piper, KittenTTS, NeuTTS, Edge TTS como padrão) ou pagos na nuvem (ElevenLabs, OpenAI, Google Gemini, MiniMax, Mistral, xAI, DeepInfra). Controle de velocidade, voz e idioma.

STT Voz vira texto

Mensagens de voz do Telegram, Discord, WhatsApp, Slack ou Signal são transcritas e injetadas direto na conversa. Provedores: Whisper local, Groq, OpenAI.

+ Por que isso importa aqui

Voz não é recurso colado por fora. É mais uma interface de entrada e saída do harness: STT alimenta o contexto como uma ferramenta, TTS é uma forma alternativa de resposta final.

Outros nomes que vão aparecer

Nome	O que é	Por que citar
AutoGPT / BabyAGI	Primeiros agentes autônomos (2023)	Provaram o conceito e provaram que loop sem freio queima dinheiro
smolagents	Biblioteca minimalista da Hugging Face	Núcleo pequeno e legível; vamos ler o código no dia 5
OpenHands	Harness focado em engenharia de software	Estado da arte em agente que edita repositório
CrewAI	Agentes com papéis e processos	Fácil de começar, difícil de depurar
AutoGen / AG2	Agentes conversando entre si	Outra escola de multiagente, forte em padrões de conversa
MCP	Protocolo para expor ferramentas	Não é harness: é o USB-C das ferramentas
LangGraph	Grafos de estado para agentes	O que usaremos a partir do dia 3

MCP não compete com harness: ele padroniza a tomada



O harness continua sendo seu. O MCP só resolve como as ferramentas se apresentam a ele, de forma padronizada.

ENTÃO POR QUE ESCREVER O NOSSO

Porque em sessenta linhas cabe tudo que esses projetos têm de essencial.

Você não vai colocar o harness da tarde em produção. Vai colocar LangGraph, e no dia 3. O que a tarde de hoje compra é a capacidade de abrir o LangGraph quando ele fizer algo estranho e saber exatamente onde olhar.

07

Quatro equipes reais

Ninguém esperou um modelo melhor. Todos construíram estrutura.

Um milhão de linhas, zero escritas à mão

3

engenheiros

5

meses

1M

linhas de código

100

linhas de AGENTS.md

O último número é o que surpreende. Eles TENTARAM a versão gigante, com toda regra e convenção do projeto, e ela piorou o resultado. A formulação que ficou: um mapa com ponteiros para documentação profunda, não um manual.

Como se escreve um arquivo de instruções

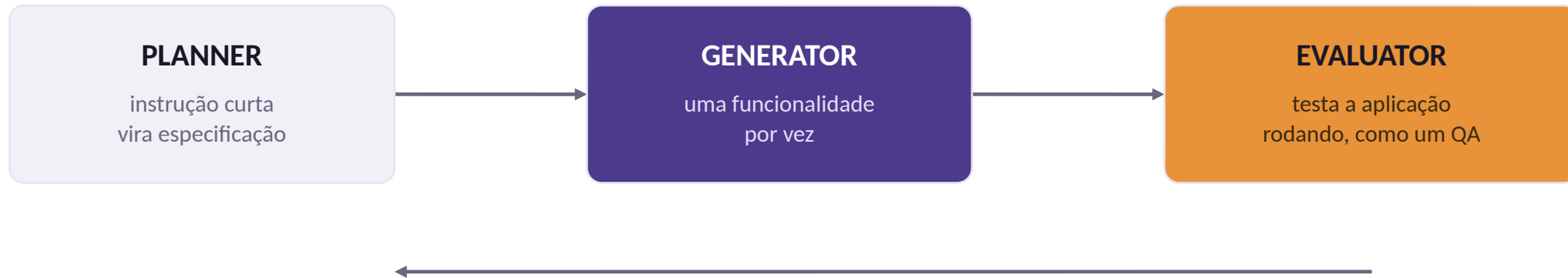
A OpenAI: dê um mapa

- Cerca de 100 linhas, deliberadamente
- Estrutura do projeto e relações entre arquivos
- Ponteiros para documentação mais profunda
- A versão gigante foi testada e piorou

B Hashimoto: um erro, uma regra

- O arquivo começa VAZIO
- Cada linha corresponde a um erro real do agente
- Cresce por indução, nunca por previsão
- Três meses depois é um harness sob medida

Deixe uma IA revisar a outra



Antes de cada ciclo, Generator e Evaluator negociam um Sprint Contract: acordam a definição de pronto antes de qualquer linha ser escrita. A seta de volta é a reprovação.

Minions: mil e trezentos pull requests por semana

1

O gargalo mudou de lugar

Quando o agente abre 1300 PRs por semana, o problema deixa de ser escrever código e passa a ser revisar código. O harness tem que produzir PR revisável.

2

Mudança pequena, escopo estreito

Cada Minion faz uma tarefa fechada. PR grande de agente não é revisado: é aprovado no escuro, e isso é pior do que não ter.

3

Verificação antes de abrir o PR

Teste, lint e build rodam no ciclo do agente. O revisor humano recebe algo que já passou nos portões automáticos.

4

O humano vira revisor, não digitador

É a mudança de papel que a turma vai viver nos próximos meses, e vale nomear agora.

Nenhuma esperou o próximo modelo

1 Instrução curta e viva

Todas mantêm um arquivo de instruções pequeno, que muda toda semana por causa de erro real, não por previsão.

2 Verificação que não depende de opinião

Teste, lint, CI, avaliador separado. Em todos os casos existe um sinal automático dizendo se está pronto.

3 Escopo pequeno por ciclo

Uma funcionalidade, um PR, uma tarefa. Ninguém deixa o agente correr solto por seis horas e depois olha o resultado.

08

Menos é mais

O princípio mais contraintuitivo do dia.

CONTEXTO NÃO É RECURSO GRÁTIS

Um harness superdimensionado é pior que harness nenhum.

Context rot: quanto mais tokens no contexto, pior o modelo recupera qualquer informação específica dentro dele. Não é um degrau que aparece no limite da janela, é degradação contínua desde o começo. Uma janela de 200 mil tokens não é um convite a usar 200 mil tokens.

Raciocínio máximo o tempo todo foi o PIOR dos três

Configuração	Nota	Observação
Raciocínio máximo o tempo todo	53,9%	Muitas tarefas estouraram o tempo
Raciocínio alto o tempo todo	63,6%	Estável, mas insuficiente
Sanduíche: máximo, depois médio, depois máximo	66,5%	A configuração escolhida

HARNESS DECAY

Conforme o modelo melhora, o harness deve encolher.

A Anthropic removeu o mecanismo de Sprint e o context reset ao migrar de Sonnet 4.5 para Opus 4.6, e trocou a avaliação por ciclo por uma única no fim. O custo caiu 38% e a qualidade não caiu. Se Agent = Model + Harness, quanto melhor o Model, menos Harness é necessário.

Três perguntas antes de manter uma linha

1 Se eu apagar, ele volta a errar?

Se a resposta é não, apague. É o único teste que importa, e é barato: apague, rode os casos guardados, compare.

2 Um bom engenheiro inferiria isso sozinho?

Se sim, o modelo também infere. Instrução só deve carregar o que é específico do seu projeto e invisível no código.

3 Isto é instrução ou é constraint?

Se dá para transformar em linter, hook ou teste, tire do texto e ponha no código. Regra em código não gasta contexto e não é ignorada.

A REGRA PRÁTICA

Construa cada peça do harness como algo que você vai apagar.

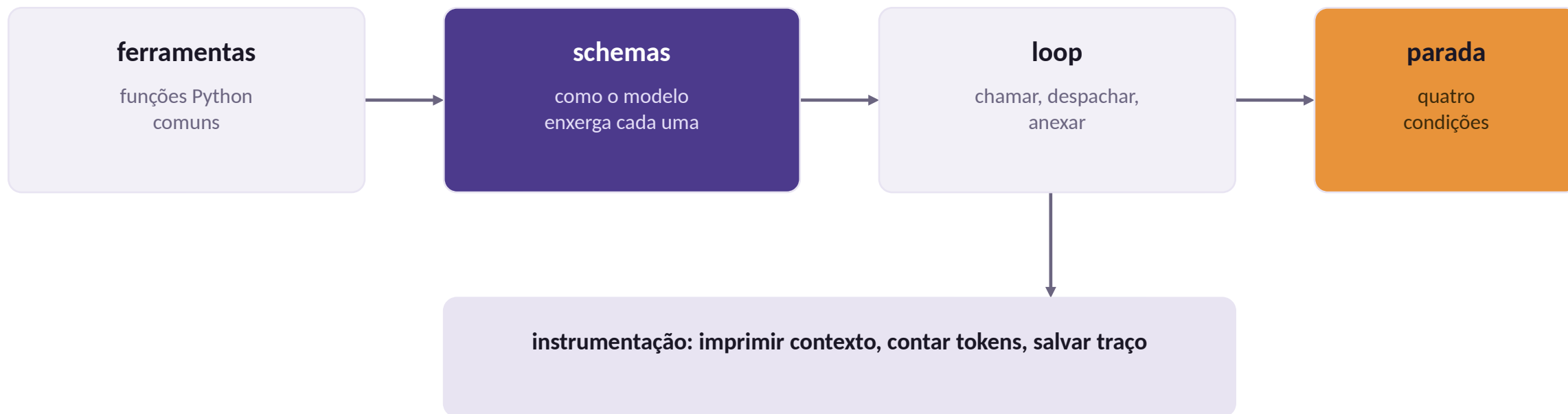
Acrescentar regra é fácil e dá sensação de progresso. Cortar é difícil, porque uma regra inútil nunca cobra a conta de forma visível: ela só encarece o contexto e piora a recuperação, em silêncio. Quem não poda acumula.

09

Escrevendo o harness

Sessenta linhas. Sem framework.

Sessenta linhas, quatro arquivos mentais



A instrumentação não é opcional: é ela que transforma um exercício que funcionou por sorte em um exercício que você consegue explicar.

Ferramentas são funções Python normais

Três decisões que já são engenharia de harness:

1. calcular NÃO usa eval(). eval() numa string vinda de um LLM é execução remota de código. O parser de AST aceita só aritmética.
2. ler_arquivo valida o caminho. Sem a checagem de prefixo, o modelo pede ../../.ssh/id_rsa e recebe.
3. Os erros voltam como texto útil, e é isso que permite ao modelo se corrigir sozinho.

```
def calcular(expressao: str) -> str:
    """Avalia uma expressao aritmetica."""
    try:
        no = ast.parse(expressao, mode="eval").body
        return str(_avaliar(no))
    except Exception as e:
        return (f"ERRO: {e}. Envie apenas numeros "
                f"e os operadores + - * / ** %.")

def ler_arquivo(caminho: str) -> str:
    alvo = (RAIZ / caminho).resolve()
    if not str(alvo).startswith(str(RAIZ)):
        return "ERRO: caminho fora da pasta."
    if not alvo.exists():
        nomes = [p.name for p in RAIZ.glob("*")]
        return f"ERRO: nao existe. Ha: {nomes}"
    return alvo.read_text()[:4000]
```

PRINCÍPIO

Uma ferramenta boa devolve, no erro, a informação de que o modelo precisa para acertar na próxima volta.

Ferramenta que só diz que falhou produz agente que repete o mesmo erro até o limite de passos. Compare: 'ERRO: arquivo não existe' contra 'ERRO: relatorio.md não existe. Disponíveis: 2023_relatorio.md, 2024_relatorio.md'. A segunda resolve sozinha na volta seguinte.

Descrever as ferramentas para o modelo

```
ESPECIFICACOES = [{  
    "type": "function",  
    "function": {  
        "name": "calcular",  
        "description": "Avalia uma expressao aritmetica e devolve o resultado exato. "  
            "Use SEMPRE que houver conta, mesmo simples.",  
        "parameters": {  
            "type": "object",  
            "properties": {  
                "expressao": {"type": "string", "description": "ex: '950 * 24'"}},  
            "required": ["expressao"]}}}]  
  
REGISTRO = {"calcular": calcular, "ler_arquivo": ler_arquivo}
```

A função de um lado, o schema JSON do outro. Guarde este incômodo: é exatamente ele que o decorador @tool do LangChain resolve amanhã.

0 loop

```
for passo in range(max_passos):                                # <- parada por limite
    resposta = cliente.chat.completions.create(
        model=modelo, messages=mensagens, tools=ESPECIFICACOES, temperature=0)
    msg = resposta.choices[0].message
    mensagens.append(msg.model_dump(exclude_none=True))          # <- contexto cresce

    if not msg.tool_calls:                                       # <- parada por resposta
        return msg.content

    for chamada in msg.tool_calls:                               # <- despacho
        args = json.loads(chamada.function.arguments)
        fn = REGISTRO.get(chamada.function.name)
        resultado = fn(**args) if fn else "ERRO: ferramenta inexistente."

        mensagens.append({"role": "tool",                        # <- observação volta
                           "tool_call_id": chamada.id,
                           "content": str(resultado)})
```

Isto é um agente. Não é brincadeira nem simplificação didática: é a mesma estrutura que existe dentro do LangChain e do smolagents.

Onde está cada componente no seu código

Componente	Onde está	Status
Montagem de contexto	a lista mensagens, que cresce a cada volta	feito
Registro de ferramentas	REGISTRO mais ESPECIFICACOES	feito, mas duplicado
Despachante	o for sobre msg.tool_calls	feito
Sandbox	validação de caminho dentro de ler_arquivo	mínimo
Condições de parada	if not tool_calls mais range(max_passos)	duas de quatro
Memória	nenhuma; some quando a função retorna	dia 3

Modelo sem tool calling nativo

Se o modelo da sua máquina não suporta o campo tools, o harness ainda funciona: muda o formato. Pede-se JSON no prompt e faz-se o parsing.

Mais frágil e mais caro em tokens, mas destrava modelos pequenos. Vale ter no bolso.

Medir a taxa de sucesso das duas abordagens no mesmo modelo é uma das extensões do exercício de hoje.

```
SISTEMA_JSON = """Responda SEMPRE com um unico
objeto JSON, sem texto ao redor.
```

```
Ferramenta:
```

```
{"acao": "calcular",
 "args": {"expressao": "2+2"}}
```

```
Resposta final:
```

```
{"acao": "final",
 "args": {"texto": "..."}}
"""
```

```
def extrair_json(texto: str) -> dict:
    i, f = texto.find("{"), texto.rfind("}")
    if i == -1 or f <= i:
        raise ValueError("sem JSON")
    return json.loads(texto[i:f + 1])
```

Laboratório da tarde: o que entregar

1

Duas ferramentas seguras

calcular sem `eval()` e `ler_arquivo` recusando caminho fora da pasta, ambas com mensagem de erro que ensina.

2

O loop completo com instrumentação

Passo, ferramenta, argumentos, 80 caracteres do resultado e tokens acumulados, impressos a cada volta.

3

As quatro condições de parada

Resposta final, limite de passos, repetição detectada e teto de tokens.

4

Três falhas provocadas em FALHAS.md

Arquivo inexistente, ferramenta inexistente e estouro do limite de passos, cada uma com causa e correção.

10

Depurar e fechar

Três instrumentos para o curso inteiro.

Monte hoje, use até o fim do curso

1 Imprimir o contexto real

Antes de cada chamada, imprima o papel e os primeiros 120 caracteres de cada mensagem. Metade dos bugs fica óbvia quando você lê o que o modelo realmente recebeu.

2 Contar tokens e passos

Some `resposta.usage.total_tokens` a cada volta. Um agente que gasta 40 mil tokens para responder quanto é `2+2` tem problema de contexto, não de modelo.

3 Guardar o traço em disco

Um JSON por execução, com todas as mensagens. É o que permite responder por que funcionou ontem e não hoje, sem depender de memória.

Como se lê um traço quando algo deu errado

1

Comece pela última mensagem antes do erro

Não pelo topo. O contexto imediatamente anterior à decisão errada é onde a causa quase sempre está.

2

Confira se a observação chegou

Procure a mensagem de papel tool correspondente a cada tool call. Faltando uma, você já achou o bug.

3

Leia o que o modelo recebeu, não o que você quis mandar

A diferença entre os dois é o bug mais frequente do curso, e é invisível se você não imprimir.

4

Compare com um traço que funcionou

Duas execuções lado a lado revelam em segundos o que a leitura de uma só não revela em meia hora.

Erros clássicos do dia 1

Sintoma	Causa provável	Correção
Tool call aparece como texto	Servidor sem parser	--enable-auto-tool-choice ou --jinja
Chama ferramenta inexistente	Descrição confusa	Nomes distintos e erro que lista as válidas
Loop infinito na mesma ferramenta	Observação não voltou	Anexar a mensagem de papel tool
Erro de tool_call_id	Chamada sem resposta	Uma resposta por tool call, sempre
Ignora a ferramenta e responde de cabeça	Description fraca	Dizer na description quando usar, não só o que faz
Resposta muda a cada execução	temperature alta	temperature=0 durante o desenvolvimento

O que o nosso harness tem, e o que falta

OK Está lá

- Loop com despacho de ferramentas
- Quatro condições de parada
- Sandbox mínima de caminho de arquivo
- Erros que ensinam o modelo
- Traço em disco e contagem de tokens

? Falta, e cada item tem um dia

- Schema derivado da função, sem duplicar (dia 2)
- Estado que sobrevive à execução (dia 3)
- Aprovação humana antes de ação irreversível (dia 3)
- Vários especialistas com contextos separados (dia 4)
- Avaliação sistemática do que foi entregue (dia 5)

As cinco camadas, e o que fizemos de cada uma hoje

Camada	O que construímos hoje	Onde aprofunda
Instructions	Prompt de sistema e description de cada ferramenta	Dia 2, com prompt templates
Constraints	Validação de caminho e limite de passos	Dia 3, com hooks e sandbox
Feedback	Mensagens de erro que ensinam o modelo	Dia 4, com agente avaliador
Memory	Nada; o histórico morre com a função	Dia 3, com checkpointer
Orchestration	Nada; um agente só, de propósito	Dia 4, com cinco topologias

PARA AMANHÃ

O nosso REGISTRO é um dicionário e o ESPECIFICACOES é uma lista escrita à mão.

O que acontece quando o sistema tem 30 ferramentas mantidas por 4 pessoas? Alguém renomeia o argumento e esquece o schema. Ninguém valida tipos. Cada um escreve o tratamento de erro de um jeito. E não há como testar uma ferramenta sem subir o agente inteiro. Amanhã: LangChain.

Três textos, cerca de quarenta minutos

1

OpenAI, Harness engineering (fev/2026)

O texto que nomeou o campo. Leia prestando atenção no tamanho do AGENTS.md e no porquê.

2

Anthropic, Effective context engineering for AI agents

Base do bloco 8 de hoje e do dia 4 inteiro. É onde context rot é medido, não apenas afirmado.

3

Yao et al., ReAct (2022)

Opcional, e curto. Ler o artigo original depois de ter escrito o loop à mão é uma experiência diferente.